



Promises e async/await

Aula 13 · Bloco 4 — Assincronismo

Por que o JavaScript não espera — e como lidar com isso

Nesta aula

1. O problema: **coisas que demoram**
2. **Promise** — o comprovante de um valor futuro
3. **async/await** — assíncrono com cara de síncrono
4. **try** / **catch** / **finally**

O problema

Buscar dados na internet **demora**.

Se o JavaScript parasse para esperar, a página inteira congelaria — nenhum clique funcionaria.

Por isso ele é **assíncrono**: dispara a tarefa demorada, **continua executando o resto**, e trata o resultado quando ele chegar.

A prova

```
console.log("1 - Pedido feito");

setTimeout(() => {
  console.log("3 - Pizza chegou! 🍕");
}, 2000);

console.log("2 - Vou assistindo TV enquanto espero");
```

A ordem da saída **não** é a ordem das linhas: **1, 2** e só então **3**.

A analogia

Você não fica parado na porta
olhando para a rua depois de pedir pizza.

Você faz outras coisas.

E a **campainha** — o callback — avisa quando chegar.

Promise — o comprovante

Um objeto que representa um valor que **ainda não existe, mas vai existir** (ou falhar).

Não é a pizza. É o comprovante que dá direito a ela.

Estado	Significa
 pending	em andamento
 fulfilled	deu certo, o valor chegou
 rejected	deu errado — rede caiu, servidor fora

Consumindo com `.then()` e `.catch()`

```
buscarUsuario(1)
  .then((usuario) => {
    console.log(`Chegou: ${usuario.nome}`); // QUANDO der certo
  })
  .catch((erro) => {
    console.log(`Falhou: ${erro.message}`); // SE der errado
  });

console.log("Esta linha roda ANTES do resultado chegar!");
```

Funciona — mas encadeado fica confuso rápido.



async / await

Código assíncrono **que se lê como síncrono**.

```
const usuario = await buscarUsuario(1);  
const pedidos = await buscarPedidos(usuario.id);
```

Sem aninhamento. Sem **.then** dentro de **.then**.

Uma sequência natural, de cima para baixo.

Na prática

```
const principal = async () => {           // async: lida com Promises
  try {
    const usuario = await buscarUsuario(1); // await: espera resolver
    console.log(`Chegou: ${usuario.nome}`);

    const pedidos = await buscarPedidos(usuario.id);
    console.log(`Pedidos: ${pedidos.length}`);
  } catch (erro) {
    console.log(`Falhou: ${erro.message}`); // qualquer reject cai aqui
  }
};
```

As três regras

1. `await` só funciona dentro de função `async`.
2. `await promessa` pausa **a função** até resolver, e devolve o **valor** — não a Promise.
3. Promise rejeitada é capturada com `try/catch` — o mesmo `try/catch` que existe em Java.

try / catch / finally

```
try {  
  const dados = await operacaoArriscada(); // o que PODE falhar  
  console.log(dados);  
} catch (erro) {  
  console.log(`Deu ruim: ${erro.message}`); // só SE falhar  
} finally {  
  console.log("Fim da operação."); // SEMPRE – com ou sem erro  
}
```

💡 O **finally** é onde você esconde o "carregando...": some da tela deu certo ou deu errado.

O que escrever e o que só saber ler

Escreva com `async/await` + `try/catch`. É o padrão do curso e do mercado.

Saiba ler `.then/.catch` — a internet está cheia deles, e você vai encontrar em código antigo o tempo todo.

⚠ Um detalhe que confunde: `await` pausa **a função** `async`, não o programa. O resto do código continua rodando normalmente.

Exercícios da aula

Na pasta `aula-13/`, rodando no Node:

1. `ex01.js` — faça a saída sair na ordem **A, C, B, D**. Preveja antes de rodar;
2. `ex02.js` — consuma `buscarUsuario` com `.then/.catch`: `id=1` e `id=99`;
3. `ex03.js` — refaça com `async/await`. Compare a legibilidade num comentário;
4. `ex04.js` — `esperar(ms)` e uma contagem regressiva 3, 2, 1, 🚀;
5. **Desafio** 🌶️ `ex05.js` — `sortearComDemora()` que rejeita se o número passar de 7.

Próxima aula

Aula 14 — Fetch e APIs

Agora que você sabe esperar,
vamos buscar dados **do mundo real**.